

Math CI/CD

Руководство по AI-ассистированным математическим исследованиям

Канонический human operating manual / SOP

Назначение: сделать математическую разработку с Lean и LLM воспроизводимой, проверяемой, понятной человеку и устойчивой к незаметному изменению утверждения, скрытым аксиомам, устаревшим review и ложным заявлениям о новизне.

Версия 0.4.3

27 сентября 2026

Нормативный источник для людей: docs/HUMAN_MANUAL.tex. Markdown-файлы репозитория являются краткими operational notes и не заменяют этот документ.

Содержание

1	Назначение и модель доверия	3
1.1	Что автоматизация может и не может гарантировать	3
2	Роли и разделение ответственности	3
3	Исследовательский объект и структура репозитория	4
4	Жизненный цикл результата	5
5	Git и GitHub: рабочий протокол	5
5.1	Ветки	5
5.2	Issue и Draft PR	6
5.3	Коммиты	6
6	Statement freeze и управление изменениями	6
7	Формализация, trusted base и зависимости	6
7.1	Placeholder proofs	7
7.2	Axiom policy	7
7.3	Exact dependency graph и review surface	7
8	Semantic review	7
9	Понимание proof и реконструкция	7
10	Assumption ablation и falsification	8
11	Литературный обзор и provenance	8
11.1	Обязательные семейства поиска	8
11.2	Provenance categories	8
12	Freshness reviews и fingerprints	8
13	Independent reproduction	9
14	CI: обязательные проверки и их смысл	9
14.1	Локальный pre-push gate	10
15	CD: scientific delivery и immutable release	10
16	Merge policy для разных научных исходов	10
17	Правила работы агентов	11
17.1	Каноническая структура prompt'ов и разделение ролей	11
17.2	Хеширование prompt-протокола и provenance запуска	12
17.3	Blind Agent Context Layer: изоляция контекста	12
17.3.1	Sanitization и пределы blindness	13
17.3.2	Привязка запуска к пакету	13
18	Incident response и sabotage testing	14
19	Definition of Done для publishable result	15
A	Краткая последовательность для новой гипотезы	15

В	Карта файлов	16
С	Команды ежедневного контроля	16

1 Назначение и модель доверия

Этот репозиторий является не только кодовой базой, но и аудиторским следом математического результата. Работа считается завершённой не тогда, когда агент написал Lean-код и не тогда, когда lake build зелёный, а когда раздельно собраны и актуальны доказательства пяти разных утверждений.

Principle

Никогда не объединяйте в один вопрос:

1. принимает ли Lean proof term;
2. соответствует ли Lean statement математике, которую команда намеревалась доказать;
3. понимает ли человек ключевые переходы доказательства;
4. является ли результат новым относительно литературы;
5. воспроизводится ли результат независимо из чистого checkout.

Зелёный Lean build отвечает только на первый вопрос относительно закодированных определений и доверенной базы.

Финальный статус VERIFIED_RESULT не присваивается LLM и не эквивалентен успешной компиляции. Это состояние governance, которое допускается только после согласования machine evidence и требуемых human reviews.

1.1 Что автоматизация может и не может гарантировать

Слой	Автоматизация хорошо проверяет	Требуется содержательной человеческой проверки
Lean/kernel	build, placeholders, зарегистрированные declarations, зависимости, trusted-base policy	адекватность определения и научный смысл формализации
Statement control	frozen hash, semantic diff, drift	является ли новая формулировка правильной постановкой задачи
Proof evidence	fingerprints, stale review, dependency surface	понимание нетривиальных переходов и математическая ценность
Literature	наличие воспроизводимого журнала, provenance fields	полнота поиска, эквивалентность/приоритет, корректность novelty language
Reproduction	commit/environment/evidence binding	независимость reproducer и отсутствие скрытого generator context

2 Роли и разделение ответственности

Один человек может выполнять несколько ролей в маленькой группе, но роли должны оставаться концептуально раздельными. Нельзя превращать авторство proof в автоматическое semantic, novelty или final approval.

Роль	Ответственность
Research Owner	Формулирует научный вопрос, область применимости, intended definitions и interpretation. Регистрирует гипотезу до длительного proof search. Не усиливает assumptions молча, когда proof search затрудняется.
Formalizer	Переводит intended statement в Lean, поддерживает buildability, документирует нетривиальные formalization choices. Может предложить изменение statement, но не менять frozen mathematics скрытно.
Skeptic / adversarial reviewer	Ищет контрпримеры, вырожденные случаи, лишние assumptions, circularity, tautological definitions, более сильные известные результаты и пути тривиализации.
Proof-understanding reviewer	Восстанавливает обычное математическое доказательство, выделяет ключевые переходы и проверяет, что proof не только существует как term, но понятен.
Literature / provenance reviewer	Пытается опровергнуть novelty; фиксирует запросы, индексы, кандидатов, citation chasing, equivalence checks и proof fingerprints.
Independent reproducer	Работает из чистого checkout на фиксированном commit/environment, желательно без приватного контекста генератора.
Final reviewer	Проверяет соответствие claim language накопленному evidence и отсутствие stale reviews.
LLM / agents	Могут предлагать conjectures, Lean code, counterexamples, queries, dependency summaries и critiques. Не имеют права имитировать human sign-off, объявлять novelty доказанной или менять frozen statement без явного SCR.

3 Исследовательский объект и структура репозитория

Каждое существенное утверждение получает неизменяемый идентификатор вида H-0042. Один H-ID соответствует одному научному объекту. Материально другое утверждение получает новый H-ID, а связь фиксируется в metadata.

```
researchctl new H-0042 "Scaling rigidity candidate"
git switch -c hyp/H-0042-scaling-rigidity
```

Каноническая карточка: hypotheses/H-0042.yaml. Связанное evidence хранится под тем же ID:

```
lit/H-0042.md
proofs/H-0042.md
provenance/H-0042.yaml
ablations/H-0042/
reviews/H-0042/
reports/.../H-0042.*
```

Attention

Markdown в lit/ и proofs/ допустим как рабочий журнал и краткая реконструкция. Нормативные инструкции для людей должны жить в LaTeX-документации. Machine-readable состояние хранится в YAML/JSON; Lean остаётся источником формальной истины.

4 Жизненный цикл результата

Нормальный путь:

CANDIDATE → SPEC_DRAFT → SPEC_FROZEN → FORMALIZED →
SEMANTIC_REVIEWED → PROOF_RECONSTRUCTED → NOVELTY_REVIEWED →
INDEPENDENTLY_REPRODUCED → VERIFIED_RESULT

Статус	Научный смысл
CANDIDATE	Есть вопрос/гипотеза; точная формализация может быть неполной. Фиксируются intended phenomenon, scope, assumptions, известные связи и первые falsification targets.
SPEC_DRAFT	Стабилизируются definitions, quantifiers, domains и assumptions. Это правильное место для semantic redesign.
SPEC_FROZEN	Точный Lean statement заморожен и получает SHA-256 snapshot. Изменение binders/assumptions/conclusion после этого является scientific change, а не refactor.
FORMALIZED	Полный Lean proof существует без запрещённых placeholders и с допустимой trusted base. Не означает meaningful/novel/understood.
SEMANTIC_REVIEWED	Человек проверил соответствие Lean statement intended mathematics и отсутствие скрытой тривиализации.
PROOF_RECONSTRUCTED	Человек понимает ключевые переходы, проверил circularity/trivialization risks и review surface.
NOVELTY_REVIEWED	Literature/provenance audit достаточно полон для заявляемого уровня novelty; поиск воспроизводим.
INDEPENDENTLY_REPRODUCED	Codebase reproducer воспроизвёл результат из чистого checkout/environment и связал review с конкретными fingerprints.
VERIFIED_RESULT	Все требуемые evidence свежи, sign-offs выполнены, policy проходит, final claim language утверждён.

Допустимые terminal/side outcomes: REFUTED, KNOWN_RESULT, SPECIFICATION_BUG, BLOCKED, INCONCLUSIVE. Полезные отрицательные результаты следует сохранять и merge'ить, а не удалять.

5 Git и GitHub: рабочий протокол

5.1 Ветки

```
hyp/H-0042-short-name    # hypothesis/proof development
formal/H-0042-short-name  # formalization
skeptic/H-0042-short-name # counterexamples/adversarial work
lit/H-0042-short-name     # literature/provenance
proof/H-0042-short-name   # human proof reconstruction
scr/H-0042-short-name     # statement change request
infra/short-name          # CI/CD infrastructure
```

Не используйте долгоживущие персональные ветки. H-ID должен присутствовать в branch/PR, изменяющем scientific object.

5.2 Issue и Draft PR

Issue создаётся до устойчивой работы и содержит H-ID, informal claim, scope, assumptions/definitions, мотивацию, known related results, counterexample targets, owners/reviewers и текущий статус. Draft PR открывается рано и становится общей coordination surface для людей и агентов.

Каждый серьёзный PR должен отвечать: что изменилось математически; frozen ли statement; какие evidence изменились; какие reviews стали stale; что доказано/опровергнуто; что искалось в литературе; какие checks необходимо повторить.

5.3 Коммиты

Предпочтительны маленькие аудируемые commits:

```
H-0042: register scaling rigidity candidate
H-0042: freeze theorem statement
H-0042: prove compactness bridge lemma
H-0042: add counterexample search results
H-0042: record nearest prior-art candidates
H-0042: reconstruct proof skeleton
infra: reject invalid Lean references at candidate stage
```

Не смешивайте изменение CI/CD и headline mathematical proof в одном commit без необходимости.

6 Statement freeze и управление изменениями

Перед freeze три Lean reference fields должны быть различены:

```
statement:
  lean_file: MathCI/Scaling.lean
  lean_module: MathCI.Scaling
  lean_declaration: MathCI.scaling_rigidity
```

lean_file — путь файловой системы; lean_module — импортируемый Lean module; lean_declaration — fully-qualified theorem/definition.

```
researchctl validate
researchctl freeze H-0042
researchctl semantic-diff H-0042
```

После freeze proof refactoring разрешён при неизменном statement. Любое semantic изменение требует явного Statement Change Request: вернуть объект в SPEC_DRAFT, объяснить причину, заморозить новую версию и повторить downstream reviews.

Forbidden / blocking condition

Запрещено менять assumption, domain, binder, conclusion или definition только для того, чтобы агент смог завершить proof, а затем выдавать это за proof исходной гипотезы.

7 Формализация, trusted base и зависимости

Во время Lean-разработки запускайте:

```
lake build
researchctl validate
```

```
researchctl semantic-diff H-0042 --fail-on-change
```

Перед FORMALIZED:

```
researchctl audit H-0042
researchctl axioms-all
researchctl deps-exact H-0042
researchctl review-surface H-0042
```

7.1 Placeholder proofs

Lean может принять sorry с warning. Math CI/CD обязан трактовать sorry/admit как hard failure. Успешный lake build сам по себе не означает complete proof.

7.2 Axiom policy

Trusted base задаётся allowlist policy. Project-specific axiom является научным предположением и должен быть явно обоснован. Нельзя добавлять axiom в allowlist только для того, чтобы CI стал зелёным.

Fresh transitive traversal используется как основной gate; #print axioms сохраняется как диагностическое сравнение.

7.3 Exact dependency graph и review surface

Exact graph строится из elaborated Lean environment. Он помогает найти project lemmas, на которых концентрируется человеческий review. Граф не заменяет математическое понимание и не является оценкой научной важности.

8 Semantic review

После готовности evidence:

```
researchctl fingerprint H-0042
researchctl review-init H-0042
```

Semantic reviewer обязан проверить definitions, assumptions, quantifiers/domains/boundaries, conclusion, vacuous/degenerate cases и отсутствие скрытого усиления спецификации. Review связан с frozen statement hash; изменение statement делает его stale/invalid.

Checklist

Перед SEMANTIC_REVIEWED: definitions соответствуют intended objects; assumptions явно перечислены; conclusion совпадает с human claim; degenerate cases рассмотрены; theorem не стал тривиальным из-за encoding; reviewer не является LLM-подписью.

9 Понимание proof и реконструкция

proofs/H-0042.md является рабочей human-readable реконструкцией, а не line-by-line переводом Lean. Она должна содержать conventional statement, 2–10 ключевых lemmas/transitions, границу routine library machinery и project-specific reasoning, использование assumptions и потенциальные circularity/trivialization points.

```
researchctl deps-exact H-0042
researchctl provenance-init H-0042
```

researchctl review-surface H-0042

Proof review привязывается к formalization fingerprint. Изменение proof source, proof reconstruction или relevant environment делает старый review stale.

10 Assumption ablation и falsification

researchctl ablate H-0042

Для каждого содержательного assumption попытайтесь удалить/ослабить его, построить counterexample без него или указать точный proof step, где он необходим. Нельзя называть assumption необходимым только потому, что текущий proof его использует.

Adversarial review должен проверять empty domains, zero/one limits, symmetry-degenerate и boundary cases, equivalent reparameterizations, tautological definitions и stronger known results.

11 Литературный обзор и provenance

Principle

Цель literature review — не подтвердить novelty, а активно попытаться её опровергнуть.

11.1 Обязательные семейства поиска

Ищите exact terminology; стандартные термины вместо project names; equivalent formulations; stronger results; weaker/general/special cases; characteristic proof ingredients; необычную последовательность reductions (proof fingerprint); источники major imported/library lemmas.

Для каждого запроса фиксируйте дату, database/index/source, воспроизводимый query, candidate results и классификацию каждого серьёзного кандидата как equivalent/stronger/weaker/orthogonal/unresolved. Для сильных кандидатов делайте backward и forward citation chasing, когда применимо.

11.2 Provenance categories

PROJECT_NEW, PROJECT_PRIOR, MATHLIB, EXTERNAL_PUBLISHED, STANDARD_FACT, UNKNOWN. UNKNOWN допустим во время исследования, но не должен незаметно пережить сильный novelty claim.

Agents могут сообщать: “matching result не найден в перечисленных источниках”. Они не должны превращать это в “theorem new”. При найденном эквивалентном prior result используйте KNOWN_RESULT.

12 Freshness reviews и fingerprints

researchctl report H-0042
researchctl fingerprint H-0042

Report должен различать PASS / VALID, PASS / STALE, MISSING, INCONCLUSIVE. STALE означает, что approval относился к старой версии evidence.

Forbidden / blocking condition

Никогда не “освежайте” `*_sha256_reviewed` механически. Правильное действие при STALE — повторно проверить изменившийся материал и только затем записать новый review hash.

Fingerprint domains должны различать statement, formalization, literature/provenance, environment и complete evidence. Это позволяет повторять только затронутые проверки, не стирая независимые approvals без причины.

13 Independent reproduction

Reproducer предпочтительно работает в чистом clone и без generator chat history:

```
git clone <repo> clean-reproduction
cd clean-reproduction
git checkout <commit>
python -m pip install -e '[dev]'
lake build
researchctl validate
researchctl axioms-all
researchctl deps-exact H-0042
researchctl env-lock
researchctl audit H-0042
```

Фиксируются exact commit, environment и способность восстановить human proof без скрытого контекста генератора.

14 CI: обязательные проверки и их смысл

Check	Что он означает
protocol-gate	Metadata, valid Lean references, statement freeze, status requirements, freshness требуемых reviews, placeholders, provenance и final policy.
unit-tests	Тестирует реализацию researchctl. Не тестирует вашу математику напрямую.
acceptance-suite	Запускает sabotage regressions: baseline, sorry, invalid Lean references, statement drift, stale proof review и другие control failures.
agent-protocol	Проверяет полноту canonical prompt files, вычисляет общий/role SHA-256 и публикует manifest как CI artifact.
blind-context	Строит изолированные role-specific context packets, проверяет visibility profiles и сканирует пакеты на утечки запрещённого глобального контекста.
workflow-audit	Проверяет опасные GitHub Actions patterns. Privileged execution untrusted PR code с secrets является ошибкой.
lean-build	Показывает, что Lean принимает код с зарегистрированным toolchain. Не доказывает novelty/meaning.
axiom-audit	Делает transitive trusted-base traversal и применяет project axiom allowlist.
exact-dependencies	Строит elaborated dependency evidence и environment artifacts.

PR mergeable только когда required machine checks зелёные и требуемые human reviews удовле-

творены.

14.1 Локальный pre-push gate

```
pytest -q
researchctl acceptance
researchctl agent-manifest
researchctl context-profiles
researchctl validate
researchctl workflow-audit
lake build
researchctl axioms-all
researchctl deps-exact-all
```

Для изменённого H-ID:

```
researchctl semantic-diff H-0042
researchctl fingerprint H-0042
researchctl report H-0042
researchctl audit H-0042
```

15 CD: scientific delivery и immutable release

В обычном software CI/CD delivery означает deploy. Здесь delivery означает выпуск неизменяемого, воспроизводимого research artifact.

Перед release:

```
researchctl validate
researchctl audit H-0042
researchctl fingerprint H-0042
researchctl packet H-0042
```

Verification packet должен содержать hypothesis card, Lean source, statement snapshot, proof reconstruction, literature/provenance evidence, reviews, dependency reports, environment lock, canonical agent prompt files, visibility profiles, blind-context input manifests, agent protocol manifest, provenance записей конкретных agent runs и SHA-256 manifest.

Research-release workflow запускается только для VERIFIED_RESULT и должен быть защищён GitHub Environment с human approval. Рекомендуемый immutable tag:

```
result/H-0042/<statement-hash-prefix>
```

Опубликованный tag не переписывается. Материальное изменение evidence создаёт новый release с явным описанием связи со старым.

16 Merge policy для разных научных исходов

Successful theorem: merge на соответствующей стадии, без прыжка сразу в VERIFIED_RESULT.

Refuted hypothesis: merge воспроизводимого counterexample/refutation и статус REFUTED.

Known result: merge provenance/literature finding и статус KNOWN_RESULT.

Specification bug: сохранить факт ошибочного encoding и поставить SPECIFICATION_BUG; не переписывать историю.

Blocked/inconclusive: документировать причину и условие разблокировки.

17 Правила работы агентов

Перед proof-agent передавайте H-ID, frozen statement или явный SPEC_DRAFT, список definitions которые можно/нельзя менять, axiom policy, имеющийся provenance/literature evidence и прямой запрет silent assumption strengthening.

Требуйте от proof-agent output в порядке: (1) interpretation exact statement; (2) proof architecture; (3) новые lemmas; (4) Lean implementation; (5) dependency/provenance notes; (6) явный запрос на specification change, если proof требует изменения постановки.

Literature-agent обязан давать reproducible queries и candidate prior art, а не бинарный novelty verdict. Skeptic-agent получает задачу искать counterexamples, удалять assumptions, проверять degenerate cases и искать stronger known results.

Forbidden / blocking condition

Агенту запрещено: менять frozen statement без SCR; самостоятельно ставить VERIFIED_RESULT; подписывать human reviews; придумывать citations/search logs; скрывать неудачу proof через усиление assumptions; трактовать green CI как научное одобрение.

17.1 Каноническая структура prompt'ов и разделение ролей

Нормативные prompt'ы агентов хранятся не в одном общем документе, а в каталоге agents/. Это часть воспроизводимой исследовательской среды.

```
agents/
  common/
    SYSTEM.md
    OUTPUT_CONTRACT.md
  formalizer/
    SYSTEM.md
    TASK.md
    OUTPUT_SCHEMA.yaml
  skeptic/
  semantic-reviewer/
  proof-reviewer/
  literature-reviewer/
  provenance-auditor/
  reproducer/
  red-team/
  merge-gate/
```

Каждая роль получает общий system contract, свой role-specific system prompt, task template и machine-readable output schema. Один универсальный агент не должен одновременно генерировать гипотезу, доказывать её, проверять семантику, подтверждать novelty и одобрять merge: разделение ролей является частью защиты от self-review.

Роль	Основная обязанность
formalizer	Формализовать ровно frozen statement; при необходимости semantic change остановиться и запросить SCR.
skeptic	Пытаться опровергнуть claim, находить boundary cases, лишние assumptions и trivialization.
semantic-reviewer	Сопоставить Lean definitions/statement с intended mathematics.
proof-reviewer	Восстановить человеческое доказательство и минимальный review surface.
literature-reviewer	Фальсифицировать novelty через воспроизводимый prior-art search.
provenance-auditor	Классифицировать происхождение содержательных зависимостей.
reproducer	Независимо воспроизвести build/evidence из чистой среды.
red-team	Автоматизированно искать блокирующие риски; не имеет права выдавать научное одобрение.
merge-gate	Проверить только repository evidence и policy gates перед merge.

17.2 Хеширование prompt-протокола и provenance запуска

Перед серьёзной AI-assisted работой создайте cryptographic manifest:

```
researchctl agent-manifest
```

Команда вычисляет общий agent_protocol_sha256 и отдельный hash каждой роли. Этот hash входит в research fingerprint и verification packet. Изменение канонического prompt'a поэтому является наблюдаемым изменением исследовательской среды.

Для подготовки prompt'a конкретной роли:

```
researchctl agent-prompt skeptic H-0042
researchctl agent-prompt formalizer H-0042
```

Для фиксации реального AI-запуска:

```
researchctl agent-run-init formalizer H-0042 \
--model gpt-5.6-sol
```

Создаваемая запись обязана содержать H-ID, роль, модель, commit, frozen statement hash, общий protocol hash и role prompt hash. После завершения запуска человек или automation заполняет реальные input/output artifacts и outcome. Нельзя переписывать hash в старой записи для “освежения” evidence; при изменении prompt'a или statement создаётся новый run record.

Attention

docs/PROMPTS.md является только кратким индексом. При любом расхождении нормативными для агентов считаются файлы в agents/. Human sign-off при этом по-прежнему не может быть заменён agent output schema.

17.3 Blind Agent Context Layer: изоляция контекста

Простого разделения ролей недостаточно. Если агенту дать весь checkout и только написать “не смотри на другие этапы”, он всё равно знает глобальную цель, результаты других ролей и желаемый исход. Для независимых стадий используется context firewall: агент получает физически отдельный минимальный пакет, а не репозиторий.

Principle

Независимый agent run должен быть воспроизводим как функция от явного input packet. Отсутствие доступа к скрытому контексту обеспечивается структурой входа, а не обещанием агента игнорировать файлы.

Канонические visibility profiles лежат в agents/profiles/.

```
researchctl context-profiles
researchctl context-build skeptic H-0042 \
  --profile skeptic-blind
researchctl context-build proof-reviewer H-0042 \
  --profile proof-blind
```

Пакет создаётся в reports/agent-inputs/H-XXXX/<RUN-ID>/ и содержит только разрешённые артефакты, нейтральный task и INPUT_MANIFEST.json. Manifest фиксирует profile hash, statement hash, protocol hash, included artifacts, declared exclusions, sanitization policy, leakage scan и context_packet_sha256.

Уровень изоляции	Что скрывается
Role separation	Только ответственность; общий контекст может быть известен. Слабая изоляция.
Output isolation	Outputs других агентов.
Task blindness	Желаемый глобальный исход, proof status, downstream цели.
Provenance blindness	Кто сгенерировал claim, generator reasoning, другие reviews.
Goal blindness	Максимально нейтральная локальная математическая постановка без project-specific цели.

Для skeptic-blind по умолчанию разрешены frozen statement, definitions, assumptions и notation; запрещены existing proof, proof reconstruction, Lean-success status, literature results, novelty claim, intended application и outputs других агентов. proof-blind не должен знать даже о существовании accepted formal proof: его задача — независимо вывести человеческий proof sketch из statement/definitions/assumptions.

17.3.1 Sanitization и пределы blindness

Профиль может скрыть H-ID, заменить project-specific identifiers нейтральными alias'ами и удалить configured markers. Это снижает anchoring, но не даёт логической гарантии, что модель не догадается о тематике по самой математике. Корректная формулировка аудита: “глобальная задача не была явно включена в зафиксированный input packet”.

Forbidden / blocking condition

Blind-run не считается независимым, если агент имел полный checkout, доступ к git history/search, outputs других ролей или project overview. Для blind-run не используйте symlink на repository и не давайте shell с рабочей директорией проекта. Передавайте только скомпилированный context packet.

17.3.2 Привязка запуска к пакету

```
researchctl agent-run-init skeptic H-0042 \
  --model <MODEL> \
  --profile skeptic-blind
```

Run record содержит visibility profile, путь к context packet, его SHA-256 и результат leakage scan. При изменении profile, statement или разрешённых входов создаётся новый packet/run; старые hashes не переписываются.

GitHub job blind-context строит smoke-packets для нескольких blind roles и выполняет автоматический leakage scan. Этот gate является required check вместе с agent-protocol. После merge новой версии обновите уже активный server-side ruleset командой `scripts/update_github_ruleset.sh OWNER/REPO`; bootstrap-скрипт существующий ruleset намеренно не перезаписывает.

18 Incident response и sabotage testing

Control system считается проверенным не одним зелёным build, а способностью правильно падать при атаке. После изменения infrastructure запускайте `researchctl acceptance` и полный matrix из `docs/FIRST_RUN.md`.

Минимальные sabotage scenarios:

Саботаж	Ожидаемое поведение
Добавить sorry	Lean может build'иться, но protocol gate обязан FAIL.
Изменить frozen binder/assumption	semantic diff показывает drift; validate FAIL.
Добавить custom axiom	fresh axiom audit находит её; policy FAIL без explicit allow.
Скрыть axiom через bridge lemma	transitive traversal всё равно находит axiom.
Изменить только proof	statement остаётся VALID; proof/reproduction reviews становятся STALE.
Сломать lean_file/module/declaration	validate выдаёт controlled INVALID_LEAN_REFERENCE, без traceback.
Попытаться release VERIFIED_RESULT	до CD workflow отказывает.

19 Definition of Done для publishable result

Checklist

- exact mathematical claim frozen;
- no placeholder proofs;
- clean Lean build;
- trusted-base/axiom audit passes;
- semantic review valid;
- human proof reconstruction complete;
- proof review valid;
- key dependency provenance classified;
- meaningful assumptions challenged/ablated where appropriate;
- literature search reproducibly logged;
- novelty review valid and claim language calibrated;
- independent reproduction valid;
- exact dependency report current;
- environment lock current;
- final review valid;
- required distinct human signers recorded;
- verification packet generated;
- immutable release created from reviewed commit.

Это и есть intended meaning VERIFIED_RESULT.

А Краткая последовательность для новой гипотезы

1. `researchctl new H-XXXX "..."`; создать Issue и Draft PR.
2. Заполнить intended statement, scope, definitions, assumptions и initial prior art.
3. На SPEC_DRAFT провести semantic/falsification work.
4. `researchctl validate`; `researchctl freeze H-XXXX`; проверить semantic-diff.
5. Разрабатывать proof, не меняя frozen statement скрытно.
6. Пройти build, placeholder, axiom и exact dependency checks.
7. Выполнить semantic review.
8. Восстановить proof человеком и выполнить proof review.
9. Выполнить ablation/adversarial tests.
10. Выполнить literature/provenance review с reproducible search log.
11. Независимо воспроизвести на clean checkout.
12. Проверить freshness всех reviews и final claim language.
13. Создать verification packet и immutable research release.

В Карта файлов

Путь	Назначение
hypotheses/H-XXXX.yaml	machine-readable scientific state
MathCI/*.lean	formal source
proofs/H-XXXX.md	рабочая human proof reconstruction
lit/H-XXXX.md	reproducible literature search log
provenance/H-XXXX.yaml	dependency provenance registry
reviews/H-XXXX/*.yaml	hash-bound review records
reports/statements/	frozen statement snapshots
reports/dependencies/	source/exact dependency evidence
reports/fingerprints/	domain/evidence fingerprints
research-policy.yaml	policy-as-code
docs/HUMAN_MANUAL.tex	canonical human SOP
AGENTS.md	concise machine/agent contract
agents/	canonical versioned prompts, visibility profiles and output schemas for AI roles
reports/agent-inputs/H-XXXX/	isolated blind-context packets with input manifests
agent-runs/H-XXXX/	provenance records of concrete AI-assisted runs
reports/agents/manifest.json	cryptographic manifest of the agent protocol

С Команды ежедневного контроля

```
# Repository baseline
pytest -q
researchctl acceptance
researchctl agent-manifest
researchctl validate
researchctl workflow-audit
lake build
researchctl axioms-all
researchctl deps-exact-all

# One hypothesis
researchctl semantic-diff H-0042
researchctl fingerprint H-0042
researchctl report H-0042
researchctl audit H-0042

# Evidence lifecycle
researchctl provenance-init H-0042
researchctl ablate H-0042
researchctl review-init H-0042
researchctl packet H-0042
```

Этот документ задаёт human operating procedure. При расхождении краткого Markdown summary и данного TeX manual приоритет имеет TeX manual, если policy-as-code/CI не устанавливают более строгих требований.